

Добавление элементов

Мы научились создавать статические списки, то есть списки, элементы которых известны на этапе создания. Следующий шаг – научиться добавлять элементы в уже существующие списки.

Метод `append()`

Для добавления нового элемента **в конец списка** используется метод `append()`.

Следующий программный код:

```
numbers = [1, 1, 2, 3, 5, 8, 13] # создаем список

numbers.append(21) # добавляем число 21 в конец списка

numbers.append(34) # добавляем число 34 в конец списка

print(numbers)
```

выведет:

```
[1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Обратите внимание, для того чтобы использовать метод `append()`, нужно, чтобы список был создан, при этом он может быть пустым.

Следующий программный код:

```
numbers = [] # создаем пустой список

numbers.append(1)

numbers.append(2)

numbers.append(3)

print(numbers)
```

выведет:

```
[1, 2, 3]
```

Важно: мы не можем использовать индексы для установки значений элементов списка, если список пустой. Следующий программный код:

```
numbers = [] # создаем пустой список
```

```
numbers[0] = 1
```

```
numbers[1] = 2
```

```
numbers[2] = 3
```

```
print(numbers)
```

приводит к ошибке:

```
IndexError: list assignment index out of range
```

Метод extend()

Можно также расширить список другим списком, путем вызова метода `extend()`.

Следующий программный код:

```
numbers = [0, 2, 4, 6, 8, 10]
```

```
odds = [1, 3, 5, 7]
```

```
numbers.extend(odds)
```

```
print(numbers)
```

выведет:

```
[0, 2, 4, 6, 8, 10, 1, 3, 5, 7]
```

Метод `extend()` как бы расширяет один список, добавляя к нему элементы другого списка.

Отличие между методами `append()` и `extend()` проявляется при добавлении строки к списку.

Следующий программный код:

```
words1 = ['iq option', 'stepik', 'beegEEK']
```

```
words2 = ['iq option', 'stepik', 'beegEEK']
```

```
words1.append('python')
```

```
words2.extend('python')
```

```
print(words1)
```

```
print(words2)
```

выведет:

```
['iq option', 'stepik', 'beegeek', 'python']
```

```
['iq option', 'stepik', 'beegeek', 'p', 'y', 't', 'h', 'o', 'n']
```

Метод `append()` добавляет строку `'python'` целиком к списку, а метод `extend()` разбивает строку `'python'` на символы `'p', 'y', 't', 'h', 'o', 'n'` и их добавляет в качестве элементов списка.

Удаление элементов

С помощью оператора `del` можно удалять элементы списка по определенному индексу.

Следующий программный код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
del numbers[5] # удаляем элемент имеющий индекс 5
```

```
print(numbers)
```

выведет:

```
[1, 2, 3, 4, 5, 7, 8, 9]
```

Элемент под указанным индексом удаляется, а список перестраивается.

Обратите внимание на синтаксис удаления, так как он отличается от обычного вызова метода. При удалении элементов не надо передавать аргумент внутри круглых скобок.

Оператор `del` работает и со срезами: мы можем удалить целый диапазон элементов списка.

Следующий программный код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
del numbers[2:7] # удаляем элементы с 2 по 6 включительно
```

```
print(numbers)
```

выведет:

```
[1, 2, 8, 9]
```

Мы можем удалить все элементы на четных позициях (0, 2, 4, ...) исходного списка.

Следующий программный код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
del numbers[::2]
```

```
print(numbers)
```

выведет:

```
[2, 4, 6, 8]
```

Удаление элементов

Вывод с помощью цикла for

Для вывода элементов списка **каждого на отдельной строке** можно использовать следующий код:

Вариант 1. Если нужны индексы элементов:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
for i in range(len(numbers)):
```

```
    print(numbers[i])
```

Мы передаем в функцию `range()` длину списка `len(numbers)`. В нашем случае длина списка `numbers`, равна 11. Таким образом вызов функции `range(len(numbers))` имеет вид `range(11)` и переменная цикла `i` последовательно перебирает все значения от 0 до 10. Это означает, что выражение `numbers[i]` последовательно вернет все элементы

списка `numbers`. Такой способ итерации списка удобен, когда нам нужен не только сам элемент `numbers[i]`, но и его индекс `i`.

Вариант 2. Если индексы не нужны:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
for num in numbers:
```

```
    print(num)
```

Этот цикл пройдет по списку `numbers`, придавая переменной цикла `num` значение каждого элемента списка (!) в отличие от предыдущего цикла, в котором переменная цикла «бегала» по индексам списка.

Если требуется выводить элементы списка на одной строке, через пробел, то мы можем использовать необязательный параметр `end` функции `print()`:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
for num in numbers:
```

```
    print(num, end=' ')
```

Вывод с помощью распаковки списка

В Python есть удобный способ вывода элементов списка без использования цикла `for`.

Вариант 1. Вывод элементов списка через один символ пробела:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
print(*numbers)
```

Такой код выведет:

```
0 1 2 3 4 5 6 7 8 9 10
```

Вариант 2. Вывод элементов списка, каждого на отдельной строке

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
print(*numbers, sep='\n')
```

Такой код выведет:

```
0
```

```
1
```

2

3

4

5

6

7

8

9

10

Поскольку строки содержат символы, подобно тому, как списки содержат элементы, то мы можем использовать распаковку строк точно так же, как и распаковку списков.

Такой код:

```
s = 'Python'
```

```
print(*s)
```

```
print()
```

```
print(*s, sep='\n')
```

выведет:

```
P y t h o n
```

P

y

t

h

o

n